

1. Тезис

Современная разработка требует автоматизации контроля качества кода. Инструменты форматирования и линтеры создают идеальный листинг, экономя время и предотвращая ошибки. Интеграция этих инструментов в PyCharm позволяет сочетать удобство IDE с мощностью специализированных утилит.

2. Проблема ручного форматирования

Человеческий фактор в коде

- Непоследовательные отступы (пробелы vs табуляция)
- Разная длина строк
- Хаотичные импорты
- Нестандартное именование

Последствия

1. Снижение читаемости кода
2. Увеличение времени code review
3. Конфликты в команде
4. Ошибки из-за невнимательности

3. Встроенные возможности PyCharm

Основные горячие клавиши

- **Ctrl + Alt + L** — форматирование файла или выделенного блока
- **Ctrl + Alt + O** — оптимизация импортов
- **Ctrl + Alt + Shift + L** — настройка параметров форматирования

Инспекции кода

PyCharm автоматически анализирует код:

- **Жёлтые подчёркивания** — рекомендации
- **Красные подчёркивания** — синтаксические ошибки
- **Зелёные подчёркивания** — нарушения стиля

Быстрое исправление (Alt + Enter)

Наведите на проблему и нажмите Alt + Enter для:

- Добавления недостающих пробелов
- Удаления лишних строк
- Исправления именования
- Оптимизации выражений

Автоматизация при сохранении

Settings — Tools — Actions on Save:

- Reformat code
- Optimize imports
- Rearrange code

Настройки стиля кода

Settings — Editor — Code Style — Python:

- Отступы: 4 пробела (не табуляции)
- Длина строки: 79-88 символов
- Пробелы вокруг операторов
- Пустые строки между функциями

Рефакторинги

- **Shift + F6** — переименование во всём проекте
- **Ctrl + Alt + M** — извлечение метода
- **Ctrl + Alt + V** — извлечение переменной

4. Шаблоны кода в PyCharm

Встроенные Live Templates

Live Templates — это предопределённые сокращения, которые разворачиваются в готовые конструкции кода.

Основные шаблоны Python:

- **main** → `if __name__ == "__main__":`
- **class** → шаблон класса
- **def** → шаблон функции
- **for** → цикл for

- **if** → условный оператор
- **try** → блок try-except

Пример использования

Введите сокращение и нажмите Tab:

```
def + Tab
```

Преобразуется в:

```
def function_name():  
    pass
```

Создание собственных шаблонов

1. **Settings — Editor — Live Templates — Python**
2. Нажмите "+" и выберите "Live Template"
3. Настройте:
 - Abbreviation: ваше сокращение
 - Description: описание шаблона
 - Template text: код шаблона
4. Укажите, что шаблон применяется к Python файлам

Полезные пользовательские шаблоны

Шаблон для тестов:

```
def test_():  
    """Test for ."""  
    pass
```

Шаблон для класса:

```
class :  
    """Class ."""  
  
    def __init__(self):  
        pass
```

5. Сторонние инструменты качества

Почему нужны дополнительные инструменты?

- Разные настройки PyCharm у разработчиков
- Нет стандартизации между разными программами
- Нельзя использовать PyCharm на сервере
- Встроенные проверки можно отключить

Black — автоматический форматтер

Философия: "Единственный правильный стиль"

Основные особенности:

- Форматирует весь код одинаково
- Почти не требует настроек
- У всех разработчиков одинаковый результат
- Не нужно спорить о стиле

Пример работы:

```
# До Black
def example(a,b,c):return a+b*c

# После Black
def example(a, b, c):
    return a + b * c
```

Flake8 — комбинированный линтер

Что проверяет Flake8:

- Правила PEP 8
- Неиспользуемые переменные
- Сложные функции
- Ошибки в коде

Три части Flake8:

1. `pycodestyle` — проверка стиля
2. `pyflakes` — поиск ошибок
3. `mccabe` — анализ сложности

`isort` — сортировщик импортов

Что делает isort:

1. Группирует импорты
2. Сортирует по алфавиту
3. Удаляет дубликаты
4. Совместим с Black

Группировка импортов:

- Стандартная библиотека
- Сторонние библиотеки
- Ваши модули

6. Практический пример

Исходный неформатированный код:

```
import sys,json
def process_data(input):return input*2
class DataHandler:def __init__(self,data):self.data=data
```

После встроенного форматирования PyCharm:

```
import sys, json

def process_data(input):
    return input * 2

class DataHandler:
    def __init__(self, data):
        self.data = data
```

После применения сторонних инструментов:

```
import json
import sys

def process_data(input_value: int) -> int:
    return input_value * 2
```

```
class DataHandler:
    def __init__(self, data):
        self.data = data
```

7. Интеграция инструментов с PyCharm

Настройка External Tools

1. **Settings — Tools — External Tools**
2. Добавьте инструменты:
 - **Name:** Black
 - **Program:** путь к Python
 - **Arguments:** -m black "*FilePath*"

Назначение горячих клавиш

1. **Settings — Keymap**
2. Найдите "Black" в External Tools
3. Назначьте Ctrl+Shift+B
4. Для isort назначьте Ctrl+Shift+I

Автоматическое форматирование

Settings — Tools — File Watchers:

- Добавьте Black как File Watcher
- Укажите Scope: Project Files
- Теперь код форматируется при сохранении

8. Конфигурация инструментов

Файл pyproject.toml

```
[tool.black]
line-length = 88

[tool.isort]
profile = "black"

[tool.flake8]
max-line-length = 88
ignore = "E203, W503"
```

Проверка в CI/CD

На сервере можно добавить проверки:

```
black --check .  
isort --check-only .  
flake8 .
```

9. Преимущества подхода

Для разработчика

- Экономия времени
- Автоматическое исправление ошибок
- Можно думать о логике, а не о форматировании

Для команды

- Одинаковый стиль у всех
- Быстрее code review
- Предсказуемое качество

Для проекта

- Читаемый код
- Лёгкое добавление новых людей
- Меньше ошибок

10. Внедрение в процесс

Этапы внедрения

1. Начать с встроенных возможностей PyCharm
2. Добавить Black для своих проектов
3. Использовать Black и isort в команде
4. Добавить Flake8 для серьёзных проектов

Практические рекомендации

- Настроить автоформатирование в PyCharm
- Добавлять инструменты по одному
- Использовать одинаковые настройки в команде

- Проверять код перед отправкой

11. Результаты

Ключевые показатели

- Весь код соответствует PEP 8
- Не нужно спорить о стиле
- Код проверяется автоматически
- Можно сразу публиковать код

Итоговые выводы

1. PyCharm даёт хорошую основу
2. Дополнительные инструменты делают код лучше
3. Вместе они работают эффективнее
4. Автоматизация экономит время

Заключение

Автоматическое форматирование нужно в современной разработке. PyCharm вместе с дополнительными инструментами создаёт профессиональный код без лишних усилий.

Настройте инструменты один раз — получайте хороший код всегда.